BÀI TẬP CÁ NHÂN 3 — CÔNG NGHỆ PHẦN MỀM 2026

AI-assisted Software Development & Software Testing & Quality Engineering

Họ và tên: Bùi Quang Chiến

Mã sinh viên: 23001837

Mã biến thể cá nhân: VT-075

Team ID: SE2026-T19 | **Vai trò:** Thành viên

Đề tài: GAME-2D-02 – Procedural Dungeon: roguelike có seed tái lập

Repository: https://github.com/Gumaisme2005/Keep-Fighting-And-Earning

Baseline Commit: `232f261c88ec7748f73d5ba6c912a91c517f1ce5`

Branch / PR: `feat/vt-075-reviewer-concurrency`

"Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI."

1. SPECIFICATION, ACCEPTANCE CRITERIA & INVARIANT (VT-075)

1.1. Problem Statement: Dữ liệu run lưu trữ (`GameManager.run_history` , `saved_seeds`) là bằng chứng bất biến để tái lập hầm ngục và chấm điểm run theo Seed. Trước đây chỉ có duy nhất vai trò người chơi (player) được phép ghi dữ liệu, không có cơ chế cho giám khảo/người chấm độc lập thẩm định tính hợp lệ của run mà không làm biến dạng dữ liệu gốc. Cần bổ sung vai trò **Reviewer (Thẩm định viên)** có quyền đọc và phê duyệt (approve) chính xác một lần trên đúng phiên bản (revision) đã xem, tuyệt đối **không được sửa/xóa dữ liệu gốc**. Khi có 120 yêu cầu đồng thời tranh chấp (nhiều reviewer cùng approve, hoặc vừa approve vừa edit), dữ liệu phải giữ tính nhất quán toàn vẹn.

1.2. Acceptance Criteria (Kiểm thử được):

Mã AC	Tiêu chí chấp nhận	Phương pháp kiểm chứng
AC1	Reviewer đọc được danh sách và chi tiết bản ghi. Dữ liệu trả về là snapshot độc lập; việc sửa đổi trên snapshot không làm thay đổi bản ghi gốc.	Assert nội dung đối chiếu hash SHA-256 dữ liệu gốc trước và sau khi can thiệp (`TC-01` , `TC-02`).
AC2	Reviewer cố tình gọi thao tác ghi (`add_record` , `update_note` , `remove_record` , `import_run_history`) phải bị từ chối ngay lập tức với mã `FORBIDDEN` ; dữ liệu gốc và số revision không đổi.	Negative Unit Tests + Concurrency Tests kiểm tra mã trả về và hash bộ nhớ (`TC-03` , `TC-04` , `TC-15`).
AC3	Phê duyệt (`approve`) chỉ thành công khi: `role=reviewer` , `expected_revision == record.revision` , record tồn tại và chưa từng được approve. Dữ liệu approval lưu tách riêng. Player gọi -> `FORBIDDEN` ; ID lạ -> `NOT_FOUND` ; revision cũ -> `STALE_REVISION` .	Happy-path, Negative tests cho từng ca lỗi, kiểm tra audit log và cấu trúc dữ liệu approval (`TC-06` .. `TC-09`).
AC4	Khi 120 reviewer cùng phê duyệt đồng thời 1 record, hệ thống chỉ chấp nhận đúng 1 phê duyệt (`OK`), 119 yêu cầu còn lại nhận `ALREADY_APPROVED` . Số lượng approval cuối cùng = 1.	Concurrency test với Semaphore cổng xuất phát 120 luồng (`TC-13` , `TC-13b`).
AC5	Khi phê duyệt diễn ra đồng thời với thao tác cập nhật của player, trạng thái cuối cùng phải toàn vẹn: nếu có approval thì `approval.revision == record.revision` ; không xảy ra lost update.	Concurrency test kết hợp 80 reviewer và 40 player updates đồng thời (`TC-14`).

1.3. Non-Functional Constraints (NFR):

- **NFR1 (Hiệu năng):** Mức tải kiểm chứng 120 tác vụ đồng thời; độ trễ phân vị 95 ($p95$) từ lúc phát lệnh đến khi hoàn tất không quá $3.5 ext{ giây}$.
- **NFR2 (Tính ổn định / Anti-flaky):** Test không phụ thuộc vào may rủi lịch biểu CPU; chạy lặp lại 30 lần liên tục phải cho kết quả đồng nhất 100%.
- **NFR3 (Tương thích Engine):** Tương thích hoàn toàn với Godot 4.3 (phiên bản ghim trong GitHub CI) và Godot 4.7.2 local.
- **NFR4 (Bảo mật đầu vào):** Áp dụng nguyên tắc Deny-by-default cho mọi vai trò lạ; mọi chuỗi input được làm sạch qua `sanitize_seed/sanitize_note` ; chặn DoS bộ nhớ trần 500 bản ghi.

1.4. Invariant / Contract Quan Trọng Nhất:
INV-1: Ràng buộc phê duyệt đúng phiên bản. Với mọi bản ghi, tồn tại tối đa một phê duyệt hoạt động, và nếu phê duyệt tồn tại thì `approval.revision == record.revision` . (Kèm bất biến phụ **INV-2: Dữ liệu gốc bất biến đối với mọi tác vụ của reviewer**).

2. AI-ASSISTED IMPLEMENTATION PLAN & REVIEW ĐỐI CHIẾU REPOSITORY

2.1. Hai phương án do AI đề xuất:

- **Phương án A:** Thêm thuộc tính `current_role` trực tiếp vào Autoload `GameManager.gd`, bổ sung lệnh kiểm tra role vào `save_seed_entry`, `remove_saved_seed`, `finish_run` và lưu trực tiếp approval vào mảng `run_history`.
- **Phương án B (Được chọn):** Thiết kế module độc lập `Scene/Review/review_service.gd` (kế thừa `RefCounted`) quản lý kho lưu trữ bản ghi và approvals riêng biệt; tự bảo vệ bằng `Mutex` đơn khối; cung cấp hàm import dữ liệu từ `GameManager.run_history` mà không can thiệp mã nguồn game gốc.

2.2. Review phản biện kế hoạch AI bằng Evidence thực tế từ Repository:

Giả định ban đầu của AI	Kết luận kiểm tra	Bằng chứng từ Repository thật (Evidence)
A1: Dự án có lớp mạng/API nên cần viết HTTP/API test.	**BÁC BỎ**	Lệnh `Select-String` quét toàn bộ `Scene/` và `Menu/` với pattern `HTTPRequest|WebSocket|ENet` trả về 0 kết quả (xem `00_plan_review_checks.log`). Đây là game 2D offline. Thay thế bằng integration test ở mức Service.
A2: Dùng framework GUT / gdUnit4 để chạy test và đo coverage.	**BÁC BỎ**	Repo không hề có thư mục `addons/`. Hệ thống test hiện có chạy hoàn toàn bằng script Godot tự viết qua `godot --headless <scene>`. Tuân thủ convention sẵn có của dự án.
A3: Khuyên chọn Phương án A vì "ít file hơn, tận dụng biến toàn cục".	**BÁC BỎ PHƯƠNG ÁN A (CHỌN PHƯƠNG ÁN B)**	Soi mã `GameManager.gd:7, 195, 261`: Mọi mutator đều phát tín hiệu `saved_data_changed.emit()` lên UI và ghi đè file đĩa thật `SAVE_FILE_PATH` mà không có Mutex. Bắn 120 thread vào GameManager sẽ gây crash thread-safety của Godot UI signal và làm hỏng file save thật! Module độc lập B giải quyết triệt để vấn đề này.
A4: Thread/Mutex/Semaphore chạy ổn định ở chế độ `--headless`.	**XÁC NHẬN (Có sửa đổi)**	Đã viết probe `thread_probe.gd`: 120 thread không khóa bị mất 11,173 cập nhật; có khóa bảo vệ an toàn 240,000/240,000. Cần cơ chế Semaphore cổng xuất phát để các luồng tranh chấp thực sự.
A5: Godot Engine hỗ trợ sẵn cờ tính line coverage từ CLI.	**BÁC BỎ**	Lệnh `godot --help` không có cờ coverage nào (chỉ có `--profiling`). Tự xây dựng công cụ instrument mã nguồn bằng Python (`coverage.py`).

3. RISK-BASED TEST STRATEGY & DANH MỤC 24 TEST CASES

3.1. Chiến lược kiểm thử dựa trên rủi ro: Tập trung tối đa nguồn lực vào rủi ro trọng tâm **Race Condition** trong mô hình kiểm tra-rồi-làm (Check-then-act) và rủi ro vi phạm tính toàn vẹn dữ liệu gốc. Loại bỏ E2E UI và HTTP networking không cần thiết, dồn sức vào Concurrency Testing trên 120 OS Threads thật với oracle kiểm đếm chính xác.

3.2. Bảng Danh Mục Test Cases (Bao gồm Concurrency, Negative Paths & Failure Paths):

Mã ID	Rủi ro / Yêu cầu kiểm tra	Mức test	Điều kiện tiên quyết / Dữ liệu đầu vào	Kết quả mong đợi	Độ ưu tiên
TC-01	AC1: Reviewer đọc danh sách/chi tiết	Unit	2 bản ghi có sẵn; role="reviewer" gọi list/get	Trả về `OK`, đúng 2 record, snapshot độc lập	P0
TC-02	AC1/INV-2: Snapshot isolation	Unit	Lấy snapshot, sửa trường note, xóa mảng list	Hash bản ghi gốc và kích thước kho không đổi	P0
TC-03	AC2: Reviewer sửa note (Negative)	Unit	Reviewer gọi `update_note()`	Trả về `FORBIDDEN`; dữ liệu gốc giữ nguyên	P0
TC-04	AC2: Reviewer add/remove (Negative)	Unit	Reviewer gọi `add_record`, `remove_record`	Đều trả về `FORBIDDEN`; kho không thay đổi	P0
TC-05	Deny-by-default role lạ (Negative)	Unit	Role rỗng, "admin", "Player", "root" x 7 API	100% trả về `FORBIDDEN`	P0
TC-06	AC3: Phê duyệt hợp lệ (Happy path)	Unit	Reviewer approve đúng `expected_revision=1`	Trả về `OK`; lưu approval riêng; record không đổi	P0
TC-07	AC3: Player phê duyệt (Negative)	Unit	Role="player" gọi `approve()`	Trả về `FORBIDDEN`; không sinh approval	P0
TC-08	Tham số đầu vào độc hại (Negative)	Unit	Revision <= 0, ID chứa `<script>`, Path traversal	Đều trả về `INVALID_INPUT` hoặc `NOT_FOUND`	P1
TC-09	AC3/INV-1: Approve revision cũ	Unit	Player sửa lên rev 2; Reviewer approve(rev=1)	Trả về `STALE_REVISION`; approve(rev=2) thành công	P0
TC-10	INV-1: Thu hồi khi bị sửa	Unit	Approve rev 1; sau đó Player update note	Approval cũ bị xóa; cho phép approve lại ở rev 2	P0
TC-11	AC4: Phê duyệt lần hai tuần tự	Unit	Reviewer 1 approve xong; Reviewer 2 approve tiếp	Yêu cầu 2 trả về `ALREADY_APPROVED`	P1
TC-12	Tích hợp import GameManager	Integration	Nạp mảng `run_history` thật (chứa HTML, phần tử rác)	Import lọc sạch qua sanitize; mảng gốc không đổi	P1
TC-13	AC4: 120 Reviewer approve đồng thời	Concurrency	120 thread cùng gọi `approve(rev=1)` qua cổng Semaphore	Đúng 1 `OK`, 119 `ALREADY_APPROVED`; 1 approval	P0
TC-13b	AC4: Cặp đua tối thiểu (2 luồng)	Concurrency	100 vòng x 2 luồng tranh chấp trên record mới	100% các vòng có đúng 1 `OK` và 1 `ALREADY`	P0
TC-14	AC5/INV-1: Đua Approve vs Update	Concurrency	80 Reviewer approve đua với 40 Player updates	Revision cuối = 41; approval khớp revision; không lost	P0
TC-15	NFR1: Tải hỗn hợp 120 tác vụ	Concurrency/Load	120 tác vụ hỗn hợp trên 12 records; đo latency	Bảo vệ invariant; **$p95 \le 3.5 ext{ giây}$**	P1
TC-16a	Failure-path: Lỗi lưu đĩa tuần tự	Failure path	Mô phỏng hook lưu đĩa trả `false` rồi `true`	Báo `PERSIST_FAILED`; không deadlock, cho phép thử lại	P0
TC-16b	Failure-path: Lỗi lưu đĩa đồng thời	Concurrency/Fail	120 luồng approve, 5 lần lưu đĩa đầu tiên thất bại	Đúng 5 `PERSIST_FAILED`, 1 `OK`, 114 `ALREADY`	P0

TC-17	Race condition: Tạo trùng ID	Concurrency	120 thread cùng gọi `add_record` chung 1 ID	Đúng 1 `OK`, 119 `DUPLICATE`; kho có đúng 1 record	P1
TC-18	Lost Update: Sửa đồng thời	Concurrency	120 thread cùng gọi `update_note` trên 1 record	Revision cuối = 121; 120 revision trả về khác nhau	P0
TC-19	NFR4: Làm sạch ghi chú (Negative)	Unit	Note chứa `<script>`, ký tự điều khiển, > 80 ký tự	Lọc sạch mã độc; độ dài cắt chuẩn ≤ 80	P1
TC-20	NFR4: Giới hạn trần bản ghi (Negative)	Unit	Thêm 500 bản ghi, sau đó thêm bản ghi thứ 501	Bản ghi 501 nhận `INVALID_INPUT`; kho cố định 500	P2
TC-21	Lifecycle: Player xóa bản ghi	Unit	Player gọi xóa record đã approve; kiểm tra ID sai	Xóa thành công; thu hồi approval; bắt lỗi ID lạ	P1
GUARD	Bảo vệ dữ liệu gốc repository	Guard/Integrity	So sánh SHA-256 `GameManager.saved_seeds` trước/sau	Hash giống nhau 100%; không làm bẩn repo thật	P0

4. TEST-FIRST IMPLEMENTATION: QUÁ TRÌNH ĐỔI MÀU RED $ O$ GREEN

4.1. Bản thảo v0 và Bằng chứng Thất bại Đúng Lý Do (RED Evidence):

Tại commit `14982e4`, bản nháp v0 chỉ khóa Mutex trong từng hàm accessor nội bộ riêng rẽ (`_read_record`, `_write_record`), khiến chuỗi thao tác "Đọc $ o$ Kiểm tra $ o$ Ghi" (Check-then-act) bị đan xen giữa các luồng. Kết quả chạy test trên v0 ghi nhận **7 FAILED / 14 PASSED** (xem [`01_red_v0.log`](docs/vt075/evidence/01_red_v0.log)):

```
[FAIL] TC-13 AC4 (concurrency): 120 reviewer cung approve -> dung 1 OK, 119 ALREADY_APPROVED - { "OK": 120 }
[FAIL] TC-13b AC4 (concurrency, cap toi thieu): 100 vong x 2 luong -> moi vong dung 1 OK - bad_rounds=100
[FAIL] TC-14 AC5/INV-1 (concurrency): 80 approve + 40 update dong thoi -> ... distinct=false
[FAIL] TC-17 (concurrency): 120 add_record cung id -> dung 1 OK, 119 DUPLICATE - { "OK": 120 }
[FAIL] TC-18 (concurrency): 120 update dong thoi -> revision cuoi = 121 ... - final_rev=2 distinct_revs=1
```

Phân tích: Toàn bộ 120 luồng đều vượt qua khâu kiểm tra và đồng loạt ghi đè (tất cả trả về `OK`), minh chứng rõ nét lỗi Race Condition đúng như dự kiến trước khi vá.

4.2. Review Patch và Bằng chứng Vượt qua Kiểm thử (GREEN Evidence):

Patch (commit `a89b250`, thay đổi vỏn vẹn +30/-2 dòng) bọc toàn bộ chuỗi Check-then-act của các API ghi trong một vùng khóa nguyên tử duy nhất thông qua các hàm helper `*_locked()`. Do `Mutex` của Godot 4 là Reentrant/Recursive Mutex, việc các accessor con gọi khóa lồng nhau bên trong vẫn hoàn toàn an toàn và không gây deadlock. Sau khi áp patch, toàn bộ test suite chuyển sang **GREEN (100% PASS)** (xem [`02_green_v1.log`](docs/vt075/evidence/02_green_v1.log)):

```
[PASS] TC-13 AC4 (concurrency): 120 reviewer cung approve -> dung 1 OK, 119 ALREADY_APPROVED - { "ALREADY_APPROVED": 11
[PASS] TC-14 AC5/INV-1 (concurrency): 80 approve + 40 update dong thoi -> final_rev=41 ok_updates=40 inv1=true distinct
[PERF] tasks=120 p50=69.8ms p95=90.2ms p99=93.2ms max=93.3ms wall=95.1ms (target p95<=3500ms)
[SUMMARY] VT-075 TESTS: 24 PASSED | 0 FAILED (Exit Code: 0)
```

5. ĐÁNH GIÁ SỨC MẠNH TEST SUITE: MUTATION TESTING, COVERAGE & FLAKY REVIEW

5.1. Thí nghiệm Deliberate Bug / Mutation Testing:

Chúng tôi đã triển khai công cụ sinh đột biến tự động `tests/vt075/tools/mutation_run.py` kiểm chứng 19 mutants:

- **Mutant bắt buộc M1:** Bỏ kiểm tra quyền của reviewer trong `update_note`. Được commit lên nhánh tạm `mutation/vt075-m1-remove-role-guard` (commit `ad2975b`). Test suite phát hiện và tiêu diệt ngay lập tức tại `TC-03`, `TC-05`, `TC-15`. Nhánh nộp bài hoàn toàn sạch không chứa code mutant.
- **Vòng 1 (18 mutants):** Tiêu diệt 16/18 mutants (89%). Hai mutant sống sót là `M9` (bỏ lọc mã độc ghi chú) và `M10` (bỏ giới hạn 500 bản ghi).
- **Vòng 2 (Bổ sung test):** Thêm `TC-19` và `TC-20`; cả `M9`, `M10` và `M19` đều bị tiêu diệt. **Mutation Score đạt tuyệt đối: 19/19 (100%)**.

5.2. Phân Tích Độ Phủ (Coverage Analysis):

Sử dụng công cụ instrument mã nguồn nội bộ `coverage.py` chèn đầu dò `cov_probe.gd`:

- **Line Coverage: 158 / 164 dòng (96.3%)**.
- **Branch (Arm) Coverage: 58 / 64 nhánh điều kiện (90.6%)**.
- *Nhận định:* 6 dòng chưa phủ là các nhánh thoát lỗi ngoại lệ giả lập khi I/O thất bại ở các hàm phụ (đã được bao phủ ở luồng phê duyệt chính). Độ phủ cao được kết hợp chặt chẽ cùng 100% Mutation Score và Ma trận Traceability đảm bảo chất lượng kiểm thử thực chất.

5.3. Kiểm Tra Flaky Tests (30 Vòng Lặp Liên Tiếp):

Chạy script `flaky_runs.py` 30 lần liên tục (mỗi lần kích hoạt 120 luồng đồng thời): **30/30 lần đạt ALL-GREEN (0 ca lỗi, 0 lần timeout)**. Độ trễ $p95$ dao động ổn định trong khoảng $79.4 ext{ms} - 97.8 ext{ms}$. Hệ thống cũng vượt qua thử nghiệm đảo ngược thứ tự test (`VT075_ORDER=reverse`) và xáo trộn ngẫu nhiên (`shuffle:<seed>`).

6. THÍ NGHIỆM KIỂM CHỨNG TẢI: 120 TÁC VỤ ĐỒNG THỜI & MỤC TIÊU P95 $\LE$ 3.5S

Harness kiểm chứng tải `tests/vt075/load_p95.gd` giải phóng đồng loạt 120 Thread vật lý qua Semaphore. Đo đạc thực nghiệm ghi file JSON 500 bản ghi thực tế trên đĩa (median = $0.42 ext{ms}$):

Kịch bản kiểm nghiệm	Độ trễ mô phỏng	Số mẫu đo	$p50$ (ms)	$p95$ (ms)	Max (ms)	Đánh giá ($p95 \le 3.5 ext{s}$)
1. No-delay	0 ms	2400	4.8	**9.2**	12.4	**ĐẠT (VƯỢT TRỘI)**
2. Calibrated real write	1.77 ms	2400	67.7	**192.6**	715.0	**ĐẠT (192.6ms << 3.5s)**
3. Nominal hook	300 µs	2400	521.7	**734.6**	775.5	**ĐẠT (734.6ms < 3.5s)**
4. Sensitivity 20ms	20 ms	2400	991.8	**1431.7**	1521.6	**ĐẠT**
5. Sensitivity 50ms	50 ms	600	2048.4	**2786.1**	3041.0	**ĐẠT (2.78s < 3.5s)**
6. Bão hòa (Saturation)	100 ms	600	3494.3	**5035.3**	5142.9	**VƯỢT NGƯỠNG (5.03s)**

Giới hạn trung thực: Thử nghiệm phản ánh chính xác hiện tượng tranh chấp vùng khóa (Lock Contention) trên tiến trình cục bộ, chưa bao gồm độ trễ vật lý và mất gói tin của đường truyền mạng.

7. MA TRẬN TRUY VẾT (TRACEABILITY MATRIX)

Acceptance Criterion / Risk	Test Case ID	Tập tin kiểm thử	Commit / Log	Kết quả
AC1: Đọc danh sách & snapshot	TC-01, TC-02	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**

AC2: Reviewer không được sửa/xóa	TC-03, TC-04, TC-15	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**
AC3: Phê duyệt theo đúng revision	TC-06, TC-07, TC-09	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**
AC4: Đua 120 Reviewer cùng approve	TC-11, TC-13, TC-13b	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**
AC5 / INV-1: Đua Approve vs Update	TC-10, TC-14	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**
Failure-path & Chống Deadlock	TC-16a, TC-16b	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**
NFR1: Tải 120 tác vụ, p95 <= 3.5s	TC-15, LoadHarness	`tests/vt075/load_p95.tscn`	`8d0ee13` / `09_load_120_p95.log`	**PASS**
NFR4: Input sanitization & Memory DoS	TC-05, TC-08, TC-19, TC-20	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `05_suite_after_round2.log`	**PASS**
Bảo toàn dữ liệu repo thật	GUARD	`tests/test_reviewer_concurrency.gd`	`c00ffaf` / `02_green_v1.log`	**PASS**

8. ĐOẠN VĂN SUY NGẪM KỸ THUẬT (ENGINEERING REFLECTION — 298 TỪ)

Quá trình áp dụng nghiêm ngặt chu trình kỹ nghệ phần mềm **Spec $ o$ Test $ o$ AI-assisted Change $ o$ Verify $ o$ Review** trên biến thể VT-075 đã mang lại những bài học sâu sắc về kiểm thử tự động và kiểm soát chất lượng.

Yếu tố quan trọng nhất giúp gia tăng mức độ tự tin (**confidence**) vào hệ thống là việc áp dụng phương pháp **Test-First song hành cùng Mutation Testing**. Thay vì chỉ nhìn vào chỉ số độ phủ dòng (Line Coverage đạt 96.3%), việc cố tình tiêm lỗi bắt buộc $M1$ (bỏ kiểm tra quyền của Reviewer) lên nhánh tạm thời và quan sát thấy bộ test lập tức đánh chặn thành công đã chứng minh test suite có "răng nanh" thực sự. Hơn thế nữa, bài học từ hai mutant sống sót $M9$ (bỏ lọc mã độc) và $M10$ (bỏ chặn trần bộ nhớ) ở Vòng 1 đã cảnh tỉnh tư duy: nếu không có mutation testing, lập trình viên rất dễ rơi vào ảo tưởng an toàn của con số coverage cao. Cùng với đó, việc thiết kế cổng xuất phát Semaphore cho 120 luồng đồng thời đã loại bỏ hoàn toàn tính chất hên xui của scheduler, giúp bộ test chạy ổn định qua hàng chục vòng lặp mà không gặp bất kỳ flaky test nào.

Tuy nhiên, rủi ro tồn dư (**residual risk**) vẫn chưa được chứng minh nằm ở chỗ: toàn bộ 120 tác vụ đồng thời hiện mới được kiểm chứng trên các luồng hệ điều hành nội bộ (In-Process OS Threads) chia sẻ chung một Mutex đệ quy trên một tiến trình duy nhất. Hệ thống chưa hề trải qua thử nghiệm rớt gói tin mạng (Packet Loss), phân mảnh mạng (Network Partitioning) hoặc tắc nghẽn I/O khi nhiều tiến trình độc lập cùng tranh chấp một file vật lý trên hệ điều hành.

Do đó, **bài kiểm thử tiếp theo có giá trị cao nhất** sẽ là: Xây dựng một bài kiểm thử tích hợp End-to-End phân tán (Distributed Integration Test) bọc `ReviewService` sau một HTTP microservice thực tế, sử dụng công cụ như k6 hoặc Locust để bắn tải 120 kết nối HTTP/WebSocket đồng thời từ nhiều máy client, kết hợp kiểm chứng cơ chế khóa tập trung (như Redis Redlock hoặc Optimistic Locking qua ETag/Version Header) để đảm bảo an toàn tuyệt đối khi vận hành online.

PHỤ LỤC 1: NHẬT KÝ SỬ DỤNG AI (AI USAGE LOG)

(Phần phụ lục ghi nhận tương tác trợ lý AI, không tính vào giới hạn 8 trang nội dung chính)

STT	Công cụ AI	Lời gọi / Prompt chính	Output nhận được từ AI	Đánh giá, Sửa đổi hoặc Bác bỏ
1	Antigravity (Claude 3.5 / Gemini)	"Đọc repo Godot này. Change request: thêm vai trò reviewer có quyền đọc và phê duyệt nhưng không được sửa dữ liệu gốc. Risk: race condition. Đề xuất 2 phương án và lập plan."	Đề xuất Phương án A (sửa `GameManager.gd`) và Phương án B (tạo `ReviewService.gd`). Khuyên nên chọn Phương án A.	**BÁC BỎ PHƯƠNG ÁN A.** Evidence: `GameManager.gd` tự ghi đè đĩa và emit signal UI không an toàn trên luồng phụ. Chọn Phương án B để cô lập dữ liệu kiểm thử.
2	Antigravity	"Viết script concurrency test cho 120 thread trong Godot."	Mã sinh ra dùng vòng lặp `for i in 120: t.start(worker)` và cho rằng các thread sẽ tự động tranh chấp.	**SỬA ĐỔI.** Khởi tạo tuần tự khiến các thread chạy xong trước khi luồng cuối kịp chạy. Đã sửa lại bằng cơ chế **Gated Semaphore Barrier** (`concurrency_harness.gd`).
3	Antigravity	"Tạo patch sửa lỗi race condition giữa v0 và v1."	Đề xuất bọc từng thao tác bằng `_mutex.lock()` trong wrapper `_locked()`.	**CHẤP NHẬN SAU KHI REVIEW DIFF.** Kiểm tra tính chất Recursive Mutex của Godot đảm bảo an toàn, không deadlock. Chuyển RED sang GREEN 100%.
4	Antigravity	"Sinh các ca kiểm thử mutation testing."	Đề xuất 18 mutations mô phỏng các sai sót logic.	**CHẤP NHẬN & PHÁT HIỆN SƠ HỞ.** Ở Vòng 1, M9 và M10 sống sót, chỉ ra bộ test còn thiếu kiểm tra `sanitize_note` và `MAX_RECORDS`. Đã thêm TC-19, TC-20 để diệt 100%.

PHỤ LỤC 2: 2 ĐIỂM CODE/TEST DO AI ĐỀ XUẤT TRÔNG HỢP LÝ NHƯNG KHÔNG ĐÁNG TIN NGAY

1. Phương án sửa trực tiếp Autoload GameManager.gd (Phương án A):
Biểu hiện hợp lý: Tiết kiệm số file mới, tận dụng biến toàn cục `run_history` có sẵn trong toàn bộ dự án game.
Lý do không đáng tin & bằng chứng thực tế: Soi mã nguồn `GameManager.gd:7, 195, 219, 261` cho thấy mọi mutator đều phát tín hiệu `saved_data_changed.emit()` lên Scene tree và mở file `user://roguelike_save_data.json` để ghi đè. Trong kiến trúc Godot Engine, việc phát tín hiệu UI từ 120 luồng phụ nền (Background OS Threads) sẽ gây crash bộ nhớ đa luồng (thread unsafe), đồng thời phá hủy file lưu game thật của người dùng.

2. Kỹ thuật đồng bộ luồng bằng sleep ngẫu nhiên (OS.delay_usec):
Biểu hiện hợp lý: Thường thấy trong các hướng dẫn lập trình phân tán để mô phỏng độ trễ mạng ngẫu nhiên giữa các client.
Lý do không đáng tin & bằng chứng thực tế: Dùng sleep không đảm bảo các luồng thực sự chạm vào critical section cùng một microsecond. Nếu CPU máy chấm bài bị quá tải hoặc luồng bị hệ điều hành hoãn lịch biểu, các yêu cầu sẽ dàn trải tuần tự và race condition biến mất, biến bài test thành **Flaky Test** (chạy lúc đỗ lúc trượt). Việc áp dụng Semaphore rào chắn giải phóng đồng loạt là giải pháp kỹ thuật bắt buộc để đảm bảo tính tái lập 100%.